

# Procedural Non-Euclidean Dungeon - Algorithm Report

by Danil NEsterov

## 1. Starting from the idea

The project starts from a simple design goal: generate a dungeon that is navigable as a coherent graph, while allowing the visual arrangement of rooms to be non-Euclidean. A room therefore does not own a permanent global position. Instead, every room has its own local coordinate system and is connected to other rooms through graph edges. When the player crosses a doorway, the next room is projected into the current view. This allows different paths through the dungeon to overlap or contradict each other visually without breaking the logical topology.

The system combines procedural content generation, graph algorithms, adaptive game pacing, pathfinding, and formal grammars. The main algorithms are:

- a blind digger / random graph walker for room generation;
- Dijkstra maps for exit placement and distance-to-goal information;
- an AI Director for adaptive guidance through enemies and health pickups;
- A\* style shortest-path navigation for enemy pursuit;
- a formal grammar for procedural enemy variation.

The generator is seed-based. By default, seed value 0 selects a fresh random seed for every run; entering any non-zero seed makes the same dungeon reproducible.

## 2. Blind digger / random graph walker

Dungeon generation is treated as construction of a graph rather than placement on a global grid. A room is a graph vertex, while a doorway connection is an edge. Generation begins with one room and one active "digger". The digger selects an unused connector, creates a new room, connects both rooms, and continues from the new room.

Two probabilities modify this basic walk. A **branch probability** can create an additional digger at the current room, causing the graph to grow in several directions. A **loop probability** can connect the digger to an already generated room instead of creating a new room. This changes the topology from a tree into a graph containing cycles.

The method is intentionally constructive: it builds one valid result directly instead of repeatedly generating and evaluating complete candidate maps. Connector capacity, room-size constraints, connector spacing,

no self-links, and no duplicate use of connectors are enforced during generation. Rooms are at least 5 x 5 cells including their wall ring, which guarantees an interior of at least 3 x 3 cells.

This approach is useful for the project because topology and presentation stay independent. The random walker decides *which rooms are connected*, while the non-Euclidean presentation decides *how those connected rooms are temporarily shown*. Constructive dungeon generation and related agent-based methods are discussed by Shaker et al. [1].

### 3. Dijkstra maps: exit placement and global guidance

After the room graph has been generated, the system computes shortest graph distances. All room-to-room transitions have the same cost, so the graph is unweighted in practice, although it is represented with a Dijkstra-style priority search.

First, a distance map is built from the player's starting room. The room with the largest shortest-path distance is selected as the exit. This places the goal as far away as possible in terms of required room transitions, not visual canvas distance.

A second Dijkstra map is then built from the exit to every room. The value stored for a room is therefore the minimum number of doorway transitions needed to reach the exit. This map becomes a reusable global field for decision-making. From any room, an adjacent room with a lower value is guaranteed to be closer to the exit.

Dijkstra's algorithm is appropriate here because it computes shortest paths from one source to all reachable graph vertices and does not depend on geometric coordinates [2]. That is especially important in this project because there is no globally consistent Euclidean layout.

### 4. AI Director: adaptive player guidance

The AI Director converts the Dijkstra distance field into dynamic pacing and guidance. It does not explicitly draw an arrow toward the exit. Instead, it places gameplay entities in the adjacent room that has the lowest distance-to-exit value.

When the player follows the gradient and enters a room closer to the exit, the Director places another guide entity in the next best adjacent room. If the player moves to a room with a larger Dijkstra value, this counts as a wrong-direction step. After two wrong-direction steps without progress, the Director places another entity to reinforce the intended route. Equal-distance movement is treated as neutral.

The entity type is selected according to the player's current health:

- when HP > 5, the heart-to-enemy ratio is **1:2**;
- when HP ≤ 5, the heart-to-enemy ratio is **2:1**.

A heart fully restores player HP on contact, while an enemy creates pressure and combat. This makes

guidance adaptive: a healthy player is more likely to encounter challenge, while a damaged player is more likely to receive recovery support. The Director therefore combines navigation information with a lightweight dynamic difficulty model.

Entities remain room-local. Adjacent rooms may be visible, but enemies and pickups are interactive only when their room is active. This prevents non-Euclidean visual overlap from causing unintended combat or healing.

## 5. A\* pathfinding for enemy following

Enemy movement uses room-local navigation. Each room owns an isolated navigation surface covering only its walkable interior. Doorway and perimeter cells are excluded, so an enemy cannot leave its logical room even when a doorway is visually open.

A\* is the standard heuristic extension of shortest-path graph search: it evaluates a node using the cost already travelled plus an estimate of the remaining cost. With an admissible heuristic, A\* can find an optimal path while exploring fewer nodes than uninformed search [3].

In the project, Godot's `NavigationAgent2D` and `NavigationServer2D` provide the practical path-query and path-following layer [4]. The gameplay code assigns a target position and follows successive path points inside the room's isolated navigation map. Conceptually, this fulfills the A\* pathfinding role while allowing the engine to manage the navigation mesh representation. The enemy wanders when the player is elsewhere and pursues the player only when both belong to the same logical room.

For a stricter academic implementation in which A\* must be directly visible in project code, the room interior could later be represented with `AStarGrid2D` or an explicit point graph. The current design keeps the same algorithmic purpose while delegating the navigation query to Godot.

## 6. Formal grammar for enemy variation

Enemy appearance is generated with a formal rewriting grammar. The grammar contains named non-terminal symbols such as head, body, core, tail, and legs. Each symbol has several possible productions. Generation starts from a higher-level structure and repeatedly replaces non-terminals with randomly selected alternatives until only terminal text remains.

This creates many enemy forms from a small set of reusable rules. The method separates *structure* from *variation*: every result follows the same general creature syntax, while local production choices create visual diversity. It is therefore more controllable than choosing every character independently at random.

The approach follows the general idea of formal grammars, where a finite set of production rules generates a potentially large language of valid structures [5]. Grammar-based generation is also a recognized family of procedural content generation methods [1].

## 7. How the algorithms work together

The algorithms operate at different scales. The blind digger creates the dungeon topology. Dijkstra analysis adds a global notion of progress without requiring a global room layout. The AI Director reads that distance field and converts it into adaptive gameplay events. Enemy navigation solves local movement inside one room, while the formal grammar changes enemy presentation without affecting topology or behavior.

This separation is important for the non-Euclidean concept. Global decisions use only the room graph; local movement uses only the active room's navigation surface; visual projection is allowed to change independently. As a result, the dungeon can look spatially impossible while still remaining algorithmically consistent and playable.

## 8. Evaluation and further work

The generator can be evaluated through graph metrics such as edge density, number of loops, number of high-degree branch rooms, and graph diameter. Probability sweeps can show how branching and looping affect these values. Higher loop probability generally increases connectivity and can shorten shortest-path distances, while additional branching tends to reduce long linear chains, although the exact result depends on connector limits and the room cap.

Further work would include balancing the AI Director from play-test data, persisting enemies and pickups when distant rooms unload, adding multiple enemy behavior types, and measuring how often Director interventions actually return players to the Dijkstra gradient. If required for assessment, enemy navigation could also be changed from the engine navigation abstraction to an explicit `AStarGrid2D` implementation.

## References

1. N. Shaker, A. Liapis, J. Togelius, R. Lopes, and R. Bidarra, "Constructive generation methods for dungeons and levels," in *Procedural Content Generation in Games*, Springer, 2016. [https://doi.org/10.1007/978-3-319-42716-4\\_3](https://doi.org/10.1007/978-3-319-42716-4_3)
2. E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, 1, 269-271, 1959. <https://doi.org/10.1007/BF01386390>
3. P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100-107, 1968. <https://doi.org/10.1109/TSSC.1968.300136>
4. Godot Engine documentation, "2D navigation overview" and `NavigationAgent2D`. [https://docs.godotengine.org/en/stable/tutorials/navigation/navigation\\_introduction\\_2d.html](https://docs.godotengine.org/en/stable/tutorials/navigation/navigation_introduction_2d.html)
5. N. Chomsky, "Three Models for the Description of Language," *IRE Transactions on Information Theory*, 2(3), 113-124, 1956. <https://chomsky.info/wp-content/uploads/195609-.pdf>